Enterprise Auto Loader Framework — Technical Documentation

POC: Parameterized Bronze Ingestion from ADLS Gen2 into Unity Catalog

Author: Cross-Platform Data Engineering Team**Version:** 3.1**Date:** October 2026**Status:** POC — Pending Expert Review

---

Executive Summary

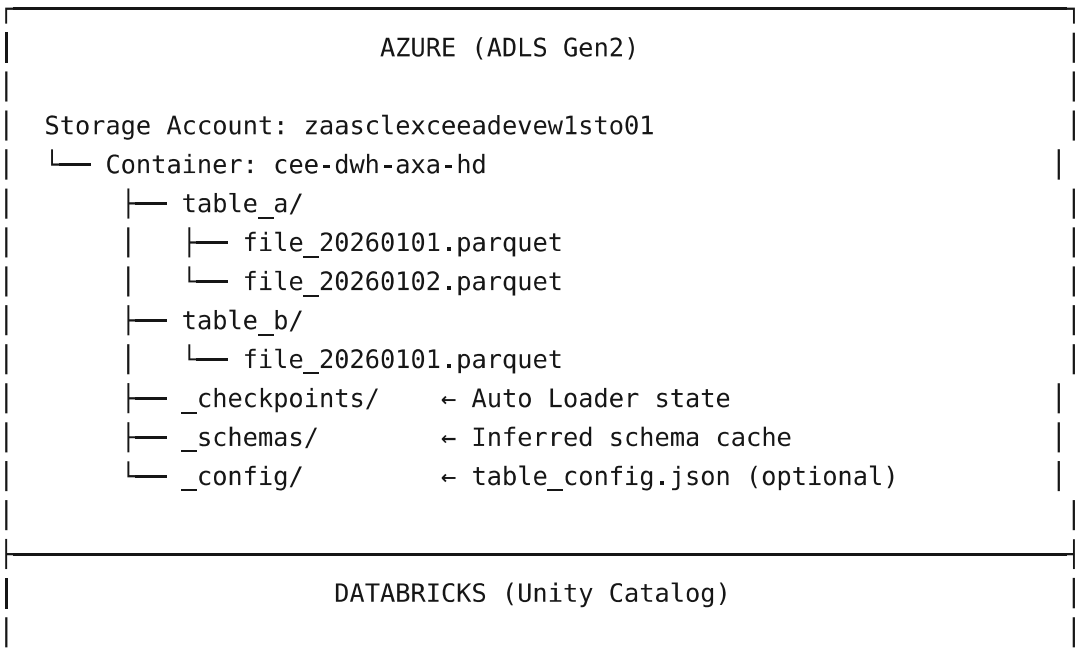
This document describes an end-to-end framework for ingesting data from Azure Data Lake Storage Gen2 (ADLS Gen2) into Databricks Unity Catalog using **Auto Loader** (Structured Streaming with `cloudFiles`). The framework is designed to be **fully parameterized**, supporting multiple teams, datasources, environments (dev/pre/pro), and load strategies — all driven by JSON configuration files.

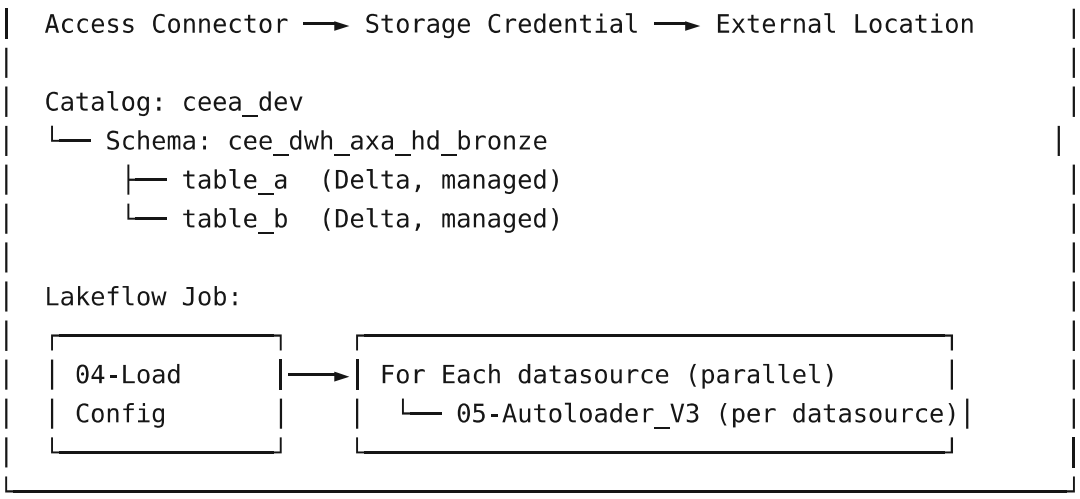
The goal is to provide a **reusable, production-grade ingestion pattern** that any team can adopt by simply providing their ADLS paths and a configuration entry.

Scope

- **14 teams**, **143 datasources**, **~12,000 tables** across the organization
- **3 environments** per team: `dev`, `pre`, `pro`
- **1 catalog per team per environment** (e.g., `cee_dev`, `cee_pre`, `cee_pro`)
- **1 schema per datasource** within each catalog (e.g., `cee_dwh_axa_hd_bronze`)
- **5 load types**: `append`, `overwrite`, `merge`, `historical_cdc`, `historical_full`
- **3 file formats**: `parquet`, `csv`, `json`

Architecture Overview





Phase 1 — Azure Infrastructure & Unity Catalog Setup

Notebook: `01 - Pre-requisits`

This notebook creates the foundational Unity Catalog objects that connect Databricks to ADLS Gen2.

1.1 Storage Credential

A single Storage Credential is shared across all storage accounts accessible by the Access Connector. This avoids credential sprawl.

```
CREATE STORAGE CREDENTIAL IF NOT EXISTS sc_npla_dva_ew1
WITH (
  AZURE_MANAGED_IDENTITY = (
    ACCESS_CONNECTOR_ID = '/subscriptions/<sub-id>/resourceGroups/z-ass-npla-dva-ew1-
rgp01/providers/Microsoft.Databricks/accessConnectors/ac-npla-dva-ew1-01'
  )
)
COMMENT 'Storage Credential for non-production data platform - West Europe';
```

Parameter	Description	Example
`ACCESS_CONNECTOR_ID`	Full Azure Resource ID of the Access Connector	`/subscriptions/.../accessConnectors/ac-npla-dva-ew1-01`
Credential name	Logical name in Unity Catalog	`sc_npla_dva_ew1`

Prerequisite: The Access Connector's Managed Identity must have the following Azure RBAC roles on the target storage accounts:

- `Storage Blob Data Contributor`
- `Storage Account Contributor` (if using file events trigger)
- `EventGrid EventSubscription Contributor` (if using file events trigger)
- `Storage Queue Data Contributor` (if using file events trigger)

1.2 External Locations

One External Location per container (i.e., per datasource). This grants Unity Catalog governed access to the ADLS path.

```
-- CEE example
CREATE EXTERNAL LOCATION IF NOT EXISTS `el_ceea_cee_dwh_axa_hd`
URL 'abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL `sc_npla_dva_ew1`)
COMMENT 'External Location for CEE team - DWH AXA HD datasource';
```

Parameter	Description	CEE Example
External Location name	Naming convention: `el_<team>_<datasource>`	`el_ceea_cee_dwh_axa_hd`
URL	`abfss://<container>@<storage_account>.dfs.core.windows.net/`	`abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/`
Storage Credential	References the credential created in 1.1	`sc_npla_dva_ew1`

Validation: After creation, verify access with:

```
LIST 'abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/';
```

1.3 Catalogs and Schemas

One catalog per team per environment. One schema per datasource within the catalog.

```
-- CEE catalog (dev environment)
CREATE CATALOG IF NOT EXISTS ceea_dev
MANAGED LOCATION 'abfss://unity-catalog-storage@<metastore-
storage>.dfs.core.windows.net/<workspace-id>/ceea_dev'
COMMENT 'CEE team - Development environment';

-- CEE schema for the DWH AXA HD datasource
CREATE SCHEMA IF NOT EXISTS ceea_dev.cee_dwh_axa_hd_bronze
COMMENT 'Bronze layer - DWH AXA HD datasource';
```

Naming convention:

Object	Pattern	CEE Example
Catalog	`<team>_<env>`	`ceea_dev`, `ceea_pre`, `ceea_pro`
Schema	`<datasource>_bronze`	`cee_dwh_axa_hd_bronze`

Table	Auto-discovered from ADLS folder name	`table_a`, `table_b`, ...

1.4 Automated External Location Discovery (Optional)

Notebook: `XX - Automatized_external_locations`

For large-scale onboarding, this notebook automatically discovers all storage accounts and containers within an Azure Resource Group and creates External Locations for each one.

Parameters:

Widget	Description	Example
`subscription_id`	Azure Subscription ID	`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`
`resource_group`	Azure Resource Group name	`z-ass-npla-dva-ew1-rgp01`
`storage_credential_name`	Storage Credential to use	`sc_npla_dva_ew1`
`auth_method`	`managed_identity` / `service_principal` / `cli`	`managed_identity`
`dry_run`	`true` = preview only, `false` = execute	`true`
`exclude_containers`	Containers to skip	`$logs,$web,$blobchangefeed`

Flow: Authenticates to Azure → Lists storage accounts in the resource group → Lists containers per account → Generates and executes `CREATE EXTERNAL LOCATION` for each container → Validates access.

Phase 2 — Datasource Configuration

Notebook: `02 - Datasources_config`

This notebook defines the **master configuration** for all datasources. Each entry represents one datasource (one ADLS container) to be ingested. The configuration is stored as a JSON file in a Unity Catalog Volume.

2.1 Configuration Schema

Each datasource entry contains:

Field	Type	Required	Description	CEE Example
`enabled`	boolean	Yes	`true` = active, `false` = skipped	`true`

team	string	Yes	Team owner (used for filtering in jobs)	"CEE"
schedule.type	string	Yes	"scheduled" or "file_arrival"	"scheduled"
schedule.frequency	string	Yes	"daily", "hourly", "weekly", "custom"	"daily"
schedule.days	array	Yes	Weekdays or ["all"]	["all"]
schedule.time	string	Yes	HH:MM in 24h CET	"06:30"
catalog	string	Yes	Target Unity Catalog catalog	"ceea_dev"
schema	string	Yes	Target schema	"cee_dwh_axa_hd_bronze"
source_base_path	string	Yes	ADLS Gen2 base path where table folders live	"abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/"
checkpoint_base_path	string	Yes	Where Auto Loader stores checkpoint state	"abfss://cee-dwh-axa-hd@.../_checkpoints/"
schema_location_base_path	string	Yes	Where Auto Loader persists inferred schema	"abfss://cee-dwh-axa-hd@.../_schemas/"
file_format	string	Yes	"parquet", "csv", or "json"	"parquet"
load_type	string	Yes	Default load strategy for all tables	"append"

primary_keys	string	No	Comma-separated PKs (only for merge)	""
trigger_mode	string	Yes	"availableNow" (batch) or "processingTime" (streaming)	"availableNow"
include_existing_files	string	Yes	"true" = process existing files on first run	"true"
table_config_path	string	No	Path to per-table overrides JSON	""

2.2 CEE Example Entry

```
{
  "enabled": true,
  "team": "CEE",
  "schedule": {
    "type": "scheduled",
    "frequency": "daily",
    "days": ["all"],
    "time": "06:30"
  },
  "catalog": "ceea_dev",
  "schema": "cee_dwh_axa_hd_bronze",
  "source_base_path": "abfss://cee-dwh-axa-
hd@zaasclexceeadevew1sto01.dfs.core.windows.net/",
  "checkpoint_base_path": "abfss://cee-dwh-axa-
hd@zaasclexceeadevew1sto01.dfs.core.windows.net/_checkpoints/",
  "schema_location_base_path": "abfss://cee-dwh-axa-
hd@zaasclexceeadevew1sto01.dfs.core.windows.net/_schemas/",
  "file_format": "parquet",
  "load_type": "append",
  "primary_keys": "",
  "trigger_mode": "availableNow",
  "include_existing_files": "true",
  "table_config_path": ""
}
```

2.3 Storage Location

The configuration file is uploaded to a Unity Catalog Volume:

```
volume_path = "/Volumes/poc/default/config/datasources.json"
dbutils.fs.put(volume_path, json.dumps(datasources, indent=2), overwrite=True)
```

Phase 3 — Per-Table Configuration (Optional)

Notebook: `03 - Table_config`

When a datasource needs **different load strategies per table**, a `table_config.json` file provides per-table overrides. Tables not listed in this file fall back to the datasource-level defaults.

3.1 Configuration Schema

```
{
  "default_load_type": "append",
  "default_file_format": "parquet",
  "tables": {
    "<table_folder_name>": {
      "load_type": "overwrite | merge | append | historical_cdc | historical_full",
      "file_format": "parquet | csv | json",
      "primary_keys": ["col1", "col2"],
      "csv_options": { "sep": ";", "header": "true", "encoding": "ISO-8859-1" },
      "notes": "Human-readable description of why this override exists"
    }
  }
}
```

3.2 Supported Load Types

Load Type	Behavior	Use Case	Requires PKs
`append`	Incremental. Only new files are processed (checkpoint tracks state).	Default for most tables. New data arrives as new files.	No
`overwrite`	Full replace. Overwrites the entire table with all pending files.	Daily full snapshots where the source sends a complete dump.	No
`merge`	CDC mirror. Upserts and deletes based on primary keys. Supports an optional `_operation` column for DELETE detection.	Source sends change data capture events (INSERT/UPDATE/DELETE).	**Yes**
`historical_cdc`	Historify CDC. Appends all change events as-is to Bronze. Deduplication and consolidation happen in Silver.	Need to preserve the full change history for auditing or SCD Type 2.	No

`historical_full`	Historify snapshots. Appends each full delivery as-is. Deduplication happens in Silver.	Source sends full snapshots but you need to keep every version.	No

3.3 Example: Mixed Load Types for AMER datasource

```
{
  "default_load_type": "append",
  "default_file_format": "parquet",
  "tables": {
    "dbo_tblcallquotelhistory": {
      "load_type": "overwrite",
      "notes": "Daily full snapshot with partitioned parquets"
    },
    "dbo_tblcalltypeserviceclose": {
      "load_type": "merge",
      "primary_keys": ["Id"],
      "notes": "CDC mirror - inserts, updates, deletes"
    },
    "dbo_tblcurrency": {
      "load_type": "historical_cdc",
      "notes": "Append all change events, consolidate in Silver"
    },
    "dbo_tbldiagnostic": {
      "load_type": "historical_full",
      "notes": "Append each full snapshot, deduplicate in Silver"
    }
  }
}
```

3.4 Storage Location

The per-table config is stored alongside the data in ADLS:

`abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/_config/table_config.json`

Referenced in the datasource config via the `table_config_path` field.

Phase 4 — Job Orchestration (Load Config)

Notebook: `04 - Load_config`

This notebook acts as the **orchestrator entry point** for a Lakeflow Job. It reads the datasource configuration, applies filters, and outputs a list of datasources for parallel ingestion via a **For Each** task.

4.1 Parameters

Widget	Description	Default	Example
`config_path`	Path to `datasources.json`	`/Volumes/poc/default/config/datasources.json`	—
`schedule_filter`	Filter by schedule frequency	`"daily"`	`"daily"`, `"hourly"`, `"weekly"`, `""` (all)
`team_filter`	Filter by team	`""` (all)	`"CEE"`

4.2 Filtering Logic

The notebook applies three sequential filters:

1. **Enabled filter:** Only datasources with `"enabled": true`
2. **Schedule filter:** Matches `schedule.frequency` (e.g., only `"daily"` datasources for the daily job)
3. **Team filter:** Matches `team` (e.g., only `"CEE"` for a team-specific run)

4.3 Output for For Each Task

The notebook outputs a list of parameter sets via `dbutils.jobs.taskValues`:

```
dbutils.jobs.taskValues.set(key="datasource_list", value=for_each_inputs)
```

Each element in the list maps directly to the Auto Loader notebook's widget parameters:

```
{
  "catalog": "ceea_dev",
  "schema": "cee_dwh_axa_hd_bronze",
  "source_base_path": "abfss://cee-dwh-axa-
hd@zaasclexceeadevew1sto01.dfs.core.windows.net/",
  "checkpoint_base_path": "abfss://cee-dwh-axa-hd@.../_checkpoints/",
  "schema_location_base_path": "abfss://cee-dwh-axa-hd@.../_schemas/",
  "file_format": "parquet",
  "trigger_mode": "availableNow",
  "processing_time": "5 minutes",
  "include_existing_files": "true",
  "table_name": ""
}
```

4.4 Lakeflow Job Structure

```
Lakeflow Job: "Bronze Ingestion - Daily"
│
├── Task 1: Load Config (04 - Load_config)
│   Parameters: schedule_filter=daily, team_filter=""
│   Output: datasource_list → [{CEE params}, {AMER params}, ...]
│
└── Task 2: For Each (datasource_list)
    ├── Iteration 1: 05 - Autoloader_V3 (CEE params)
```

```
├── Iteration 2: 05 - Autoloader_V3 (AMER params)
└── ... (parallel execution)
```

Phase 5 — Auto Loader Ingestion Engine

Notebook: `05 - Autoloader_V3`

This is the **core ingestion notebook**. It is called once per datasource (by the For Each task) and ingests **all** tables within that datasource.

5.1 Parameters (Widgets)

Widget	Type	Required	Description	CEE Example
`catalog`	text	**Yes**	Target Unity Catalog catalog	`ceea_dev`
`schema`	text	**Yes**	Target schema	`cee_dwh_axa_hd_bronze`
`source_base_path`	text	**Yes**	ADLS base path where table folders live	`abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/`
`checkpoint_base_path`	text	**Yes**	Checkpoint base path	`abfss://cee-dwh-axa-hd@.../_checkpoints/`
`schema_location_base_path`	text	**Yes**	Schema location base path	`abfss://cee-dwh-axa-hd@.../_schemas/`
`table_name`	text	No	Single table to ingest (empty = discover all)	`""`
`file_format`	text	No	Default source format	`parquet`
`trigger_mode`	dropdown	No	`availableNow` (batch) or `processingTime` (streaming)	`availableNow`

processing_time	text	No	Interval for streaming mode	5 minutes
include_existing_files	dropdown	No	Process existing files on first run	true
load_type	dropdown	No	Default load type	append
primary_keys	text	No	Default PKs for merge (comma-separated)	""
cdc_operation_column	text	No	Column indicating CDC operation	_operation
table_config_path	text	No	Path to per-table config JSON	""

5.2 Execution Flow

```
START
  │
  ├── 1. Validate parameters (assert required fields)
  │
  ├── 2. Discover tables
  │     └── List subfolders in source_base_path
  │         (exclude folders starting with "_" or ".")
  │
  ├── 3. Load per-table config (if table_config_path provided)
  │     └── Merge per-table overrides with global defaults
  │
  ├── 4. For each table:
  │     ├── a. Create target Delta table (IF NOT EXISTS)
  │     │      └── Column Mapping enabled (delta.columnMapping.mode = 'name')
  │     │
  │     ├── b. Build Auto Loader options (format-specific)
  │     │      ├── cloudFiles.schemaEvolutionMode = addNewColumns
  │     │      ├── rescuedDataColumn = _rescued_data
  │     │      └── Format-specific: inferColumnTypes, pathGlobFilter, csv options
  │     │
  │     ├── c. Read stream (spark.readStream.format("cloudFiles"))
  │     │
  │     ├── d. Add Bronze metadata columns:
  │     │      ├── _ingest_ts  (current_timestamp)
  │     │      └── _source_file (_metadata.file_path)
  │     │
  │     └── e. Write based on load_type:
```

```
│            ├── append         → writeStream.append.toTable()
│            ├── overwrite      → foreachBatch → write.mode("overwrite")
│            ├── merge          → foreachBatch → DeltaTable.merge()
│            ├── historical_cdc → writeStream.append.toTable()
│            └── historical_full→ writeStream.append.toTable()
│
└── 5. Summary report (success/failure counts, row totals)
```

5.3 Auto Loader Options by Format

Option	Parquet	CSV	JSON
`cloudFiles.format`	`parquet`	`csv`	`json`
`cloudFiles.inferColumnTypes`	`true`	`false`	`true`
`pathGlobFilter`	`*.parquet`	`*.csv`	`*.json`
`cloudFiles.schemaEvolutionMode`	`addNewColumns`	`addNewColumns`	`addNewColumns`
`rescuedDataColumn`	`_rescued_data`	`_rescued_data`	`_rescued_data`
`cloudFiles.schemaLocation`	✓	✓	✓
`cloudFiles.includeExistingFiles`	✓	✓	✓
`sep`	—	`;` (default, overridable)	—
`header`	—	`true` (default, overridable)	—
`encoding`	—	`ISO-8859-1` (default, overridable)	—
`multiLine`	—	—	`true`

5.4 Merge (CDC) Logic Detail

When `load_type = "merge"`, the notebook uses `foreachBatch` with Delta MERGE:

- **If `_operation` column exists in the data:**
  - `whenMatchedDelete` where `_operation = 'DELETE'`
  - `whenMatchedUpdateAll` where `_operation != 'DELETE'`
  - `whenNotMatchedInsertAll` where `_operation != 'DELETE'`
- **If `_operation` column does NOT exist:**

- whenMatchedUpdateAll
- whenNotMatchedInsertAll

The merge condition is built dynamically from the `primary_keys` list:

```
merge_cond = " AND ".join([f"target.`{k}` = source.`{k}`" for k in primary_keys])
```

5.5 Delta Table Properties

All target tables are created with:

```
TBLPROPERTIES (
  'delta.columnMapping.mode' = 'name',
  'delta.minReaderVersion' = '2',
  'delta.minWriterVersion' = '5'
)
```

Column Mapping (`name` mode) allows columns to be renamed, dropped, or reordered without rewriting data — essential for schema evolution in Bronze.

5.6 Metadata Columns Added

Every row ingested into Bronze includes:

Column	Type	Source	Purpose
`_ingest_ts`	timestamp	`current_timestamp()`	When the row was ingested into Databricks
`_source_file`	string	`_metadata.file_path`	Full ADLS path of the source file
`_rescued_data`	string	Auto Loader built-in	Captures data that doesn't match the inferred schema

5.7 Exit Codes

The notebook exits with a JSON payload for job monitoring:

```
// Success
{"status": "SUCCESS", "tables_ingested": 42, "total_rows": 1500000}

// Partial failure
{"status": "PARTIAL_FAILURE", "success": 40, "failed": 2, "failed_tables": ["table_x",
"table_y"]}
```

End-to-End Example: CEE Team Onboarding

This section walks through the complete setup for the **CEE** team's `cee-dwh-axa-hd` datasource.

Step 1: Azure Prerequisites

Item	Value
Resource Group	`z-ass-npla-dva-ew1-rgp01`
Storage Account	`zaasclexceeadevew1sto01`
Container	`cee-dwh-axa-hd`
ADLS Path	`abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/`

Step 2: Unity Catalog Objects (Notebook 01)

```
-- Storage Credential (shared, already exists)
-- sc_npla_dva_ew1

-- External Location
CREATE EXTERNAL LOCATION IF NOT EXISTS `el_ceea_cee_dwh_axa_hd`
URL 'abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL `sc_npla_dva_ew1`)
COMMENT 'External Location for CEE team - DWH AXA HD datasource';

-- Catalog and Schema
CREATE CATALOG IF NOT EXISTS ceea_dev COMMENT 'CEE team - Development environment';
CREATE SCHEMA IF NOT EXISTS ceea_dev.cee_dwh_axa_hd_bronze COMMENT 'Bronze layer - DWH AXA HD datasource';
```

Step 3: Add Datasource Config (Notebook 02)

Add the CEE entry to `datasources.json` (see Section 2.2 above).

Step 4: (Optional) Add Per-Table Config (Notebook 03)

If some CEE tables need different load strategies, create a `table_config.json` at:

`abfss://cee-dwh-axa-hd@zaasclexceeadevew1sto01.dfs.core.windows.net/_config/table_config.json`

Step 5: Run the Job

Option A — **Full pipeline via Lakeflow Job:**

- Task 1: `04 - Load_config` with `team_filter=CEE`
- Task 2: For Each → `05 - Autoloader_V3`

Option B — **Direct notebook execution (testing):**

- Run `05 - Autoloader_V3` with CEE parameters filled in the widgets.

Questions for the Databricks Expert

We would appreciate feedback on the following areas:

1. **Trigger mode:** We use `availableNow=True` for batch processing. Is this the recommended approach for daily scheduled ingestion, or should we consider `processingTime` with a short interval?
2. **Checkpoint and schema location:** We store checkpoints and schema cache in the same ADLS container as the source data (under `_checkpoints/` and `_schemas/`). Is this a best practice, or should these be stored in a separate location (e.g., a Databricks-managed volume)?
3. **Naming conventions at scale:** Our current naming pattern is: catalogs as `<team>_<env>`, schemas as `<datasource>_bronze`, external locations as `el_<team>_<datasource>`, and storage credentials as `sc_<platform>_<region>` (see Appendix B). As we scale this to all 14 teams, ~12,000 tables, 3 environments, and eventually multiple layers (bronze/silver/gold), do you see any potential issues or improvements with this naming convention? Are there Databricks-recommended naming patterns we should adopt before it becomes too costly to rename?
4. **Schema evolution:** We use `cloudFiles.schemaEvolutionMode = addNewColumns`. What happens when a column is removed from the source? Should we handle this differently?
5. **Merge strategy:** For CDC tables, we use `foreachBatch` + Delta `MERGE`. Is there a more efficient pattern for high-volume CDC ingestion?
6. **Cluster sizing:** What cluster type and size would you recommend for this workload? We are considering:
   - Serverless for the orchestrator notebook (04)
   - Job clusters with auto-scaling for the Auto Loader notebooks (05)
   - What instance types and min/max workers for ~12,000 tables across 143 datasources?
7. **Parallelism:** The For Each task runs one iteration per datasource. Within each iteration, tables are ingested sequentially. Should we add intra-datasource parallelism (e.g., using `ThreadPoolExecutor`)?
8. **Error handling:** Currently, a failed table does not stop the ingestion of other tables in the same datasource. The job reports `PARTIAL_FAILURE`. Is this the right approach, or should we implement retry logic?
9. **File events vs. directory listing:** We currently use directory listing mode. For production with ~12,000 tables, should we switch to file events (Azure Event Grid) for better performance?
10. **Column Mapping:** We enable `delta.columnMapping.mode = 'name'` on all tables. Are there any caveats or performance implications we should be aware of?
11. **Monitoring and alerting:** What monitoring patterns do you recommend for tracking ingestion health at this scale?
12. **One job per schema vs. one job for everything:** Our current design uses a single Lakeflow Job with a For Each task that iterates over all enabled datasources. We believe it might be better to have **one dedicated job/pipeline per schema** (i.e., per datasource), so each team's ingestion is isolated, independently schedulable, and failures in one datasource don't affect others. What is the recommended approach at this scale (143 datasources, ~12,000 tables)?
13. **Infrastructure provisioning — Databricks Jobs vs. DevOps IaC:** Currently, we create Unity Catalog infrastructure (Storage Credentials, External Locations, Catalogs, Schemas) manually via notebook `01 - Pre-requisits` and semi-automatically via `XX - Automatized_external_locations`. For production, should we keep this approach (Databricks notebooks/jobs that run CREATE statements), move to **Infrastructure as Code** via Azure DevOps / Terraform / Databricks Asset Bundles, or use a hybrid approach (IaC for Azure resources + Databricks notebooks for UC objects)? What is the recommended pattern for managing Unity Catalog infrastructure at scale across 3 environments (dev/pre/pro)?

Appendix A: Folder Structure in ADLS

```
<container>/
├── table_folder_1/
│   ├── file_20260101.parquet
│   ├── file_20260102.parquet
```

```
│   └── ...
├── table_folder_2/
│   └── ...
├── _checkpoints/
│   ├── table_folder_1/
│   └── table_folder_2/
├── _schemas/
│   ├── table_folder_1/
│   └── table_folder_2/
└── _config/
    └── table_config.json
```

Appendix B: Naming Conventions

Object	Pattern	Example
Storage Credential	`sc_<platform>_<region>`	`sc_npla_dva_ew1`
External Location	`el_<team>_<datasource>`	`el_ceea_cee_dwh_axa_hd`
Catalog	`<team>_<env>`	`ceea_dev`
Schema	`<datasource>_bronze`	`cee_dwh_axa_hd_bronze`
Table	Auto-discovered (ADLS folder name)	`dbo_tblcurrency`
Checkpoint path	`<source_base_path>/_checkpoints/<table>/`	`abfss://.../_checkpoints/dbo_tblcurrency/`
Schema location	`<source_base_path>/_schemas/<table>/`	`abfss://.../_schemas/dbo_tblcurrency/`

Appendix C: Configuration File Locations

File	Location	Purpose
`datasources.json`	`/Volumes/poc/default/config/datasources.json`	Master datasource registry
`table_config.json`	`<source_base_path>/_config/table_config.json`	Per-table overrides (optional)